

Bank Database – Technical Documentation

1. Introduction

Purpose

This document provides a comprehensive technical overview of the bank database's logical schema. It is intended for database designers, developers and reviewers to understand the structure, constraints, and relationships that underpin the banking system.

Scope

- Defines all tables, columns and data types
- Describes key constraints (PK, FK, UNIQUE, NULLability)
- Illustrates cardinalities and special cases (M:N bridges, 1:0..1 extensions)
- Serves as a reference for SQL DDL generation and future maintenance

The bank database is designed to support core retail-banking operations by modeling customers, accounts, transactions, cards, loans and related reference data. It comprises 29 tables organized into three main groups:

1. Core Entities

- **Customer, Account, Transaction, Card and Loan** form the heart of the system. They track who your customers are, the bank accounts they hold, every financial movement, the payment cards issued, and any credit facilities extended.

2. Bridge & Extension Tables

- **Disposition** and **CustomerLoan** implement many-to-many relationships (customers ↔ accounts and customers ↔ loans), each with a role-type lookup (DispositionRole, CustomerLoanRole).
- **SavingsAccountDetail** and **CardTransaction** extend Account and Transaction with specialized fields in strict one-to-one (0..1) relationships.

3. Reference & Lookup Tables

- Geographical lookups (**Country, Region, City, PostalCode, Address**) normalize location data and eliminate redundancy.
- Value-lists for business domains include **AccountType, Currency, CardType, TransactionType, MerchantCategory, EmployeeRole, Gender,** and **Merchant/Payee** as pay-to/pay-from entities.

All foreign-key relationships enforce data integrity and accurately model real-world cardinalities—one-to-many for most parent-child links, many-to-many via bridge tables, and optional one-to-one extensions where appropriate. This logical schema can be directly translated into a normalized SQL DDL script in third normal form, ensuring both performance and maintainability.

2. Table List

1. Customer
2. Branch
3. Address
4. PostalCode
5. City
6. Region
7. Country
8. Disposition
9. DispositionRole
10. Account
11. AccountType
12. SavingsAccountDetail
13. Currency
14. Card
15. CardType
16. CardTransaction
17. Transaction
18. TransactionType
19. Merchant
20. MerchantCategory
21. Loan
22. LoanType
23. CustomerLoan
24. CustomerLoanRole
25. LoanPayment
26. Employee
27. EmployeeRole
28. Gender
29. Payee

3. Table Specifications

3.1 Customer

Purpose: Stores basic customer information.

Column	Datatype	Description	Constraint
CustomerID	INT	Unique customer identifier	PK, Identity(1,1), NOT NULL
FirstName	VARCHAR(100)	Customer's first name	NOT NULL
MiddleName	VARCHAR(100)	Middle name (if any)	NULLABLE
LastName	VARCHAR(100)	Customer's last name	NOT NULL
NationalID	VARCHAR(50)	National ID / personal number	NOT NULL, UNIQUE
BirthDate	DATE	Date of birth	NOT NULL
Email	VARCHAR(255)	E-mail address	NULLABLE
PhoneNumber	VARCHAR(50)	Phone number	NULLABLE
CreatedDate	DATETIME	Record creation timestamp	NOT NULL
GenderID	INT	FK → Gender(GenderID)	FK, NOT NULL
AddressID	INT	FK → Address(AddressID)	FK, NOT NULL
BranchID	INT	FK → Branch(BranchID)	FK, NOT NULL

Relationer: Customer → Gender (M:1), Customer → Address (M:1), Customer → Branch (M:1), Customer → Disposition (1:N)

3.2 Branch

Purpose: Represents bank branches.

Column	Datatype	Description	Constraint
BranchID	INT	Unique branch identifier	PK, Identity(1,1), NOT NULL
BranchName	VARCHAR(200)	Branch name	NOT NULL
CreatedDate	DATETIME	Branch opening date	NOT NULL
PhoneNumber	VARCHAR(50)	Contact phone	NULLABLE
SWIFTCode	VARCHAR(20)	Branch's SWIFT/BIC code	NULLABLE
AddressID	INT	FK → Address(AddressID)	FK, NOT NULL

Relationer: Branch → Address (M:1), Branch → Customer (1:N), Branch → Employee (1:N)

3.3 Address

Purpose: Normalized address data (street + house).

Column	Datatype	Description	Constraint
AddressID	INT	Unique address identifier	PK, Identity(1,1), NOT NULL
Street	VARCHAR(200)	Street name	NOT NULL
HouseNumber	VARCHAR(20)	House or apartment number	NOT NULL
PostalCodeID	INT	FK → PostalCode(PostalCodeID)	FK, NOT NULL

Relationer: Address → PostalCode (M:1), Address → Customer/Branch/Employee (1:N)

3.4 PostalCode

Purpose: Postal code lookup.

Column	Datatype	Description	Constraint
PostalCodeID	INT	Unique postal-code ID	PK, Identity(1,1), NOT NULL
Code	VARCHAR(20)	The postal code value	NOT NULL
CityID	INT	FK → City(CityID)	FK, NOT NULL

Relationer: PostalCode → City (M:1), PostalCode → Address (1:N)

3.5 City

Purpose: City reference table.

Column	Datatype	Description	Constraint
CityID	INT	Unique city ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(200)	City name	NOT NULL
RegionID	INT	FK → Region(RegionID)	FK, NOT NULL

Relationer: City → Region (M:1), City → PostalCode (1:N)

3.6 Region

Purpose: Regions within countries.

Column	Datatype	Description	Constraint
RegionID	INT	Unique region ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(200)	Region name	NOT NULL
CountryID	INT	FK → Country(CountryID)	FK, NOT NULL

Relationer: Region → Country (M:1), Region → City (1:N)

3.7 Country

Purpose: Country lookup.

Column	Datatype	Description	Constraint
CountryID	INT	Unique country ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(200)	Country name	NOT NULL
ISOCode	CHAR(3)	ISO-3 code	NOT NULL, UNIQUE

Relationer: Country → Region (1:N)

3.8 Disposition

Purpose: Bridge table linking customers and accounts (M:N).

Column	Datatype	Description	Constraint
DispositionID	INT	Unique disposition record ID	PK, Identity(1,1), NOT NULL
CustomerID	INT	The customer	FK → Customer(CustomerID), NOT NULL
AccountID	INT	The account	FK → Account(AccountID), NOT NULL
DispositionRoleID	INT	Role in the account (Owner, Viewer)	FK → DispositionRole(DispositionRoleID), NOT NULL

Relationer: Disposition → Customer (M:1), Disposition → Account (M:1), Disposition → DispositionRole (M:1)

3.9 DispositionRole

Purpose: Defines possible roles in a disposition (e.g. Owner).

Column	Datatype	Description	Constraint
DispositionRoleID	INT	Unique role ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Role name	NOT NULL, UNIQUE

Relationer: DispositionRole → Disposition (1:N)

3.10 Account

Purpose: Stores all bank account records.

Column	Datatype	Description	Constraint
AccountID	INT	Unique account ID	PK, Identity(1,1), NOT NULL
AccountNumber	VARCHAR(50)	Account number	NOT NULL, UNIQUE
IBAN	VARCHAR(34)	International IBAN	NULLABLE, UNIQUE
SWIFTCode	VARCHAR(20)	SWIFT/BIC code	NULLABLE
Balance	DECIMAL(18,2)	Current balance	NOT NULL
CreatedDate	DATETIME	Opened timestamp	NOT NULL
AccountTypeID	INT	Account type	FK → AccountType(AccountTypeID), NOT NULL
CurrencyID	INT	Currency	FK → Currency(CurrencyID), NOT NULL

Relationer: Account → AccountType (M:1), Account → Currency (M:1), Account → Disposition (1:N), Account → SavingsAccountDetail (1:0..1), Account → Card (1:N), Account → Transaction (1:N)

3.11 AccountType

Purpose: Lookup of account types (Savings, Checking...).

Column	Datatype	Description	Constraint
AccountTypeID	INT	Unique account type ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Type name	NOT NULL, UNIQUE

Relationer: AccountType → Account (1:N)

3.12 SavingsAccountDetail

Purpose: Extra attributes for savings accounts (1-to-0..1 extension).

Column	Datatype	Description	Constraint
SavingsAccountID	INT	Unique savings detail ID	PK, Identity(1,1), NOT NULL
InterestRate	DECIMAL(5,2)	Annual interest rate (%)	NOT NULL
MinimumBalance	DECIMAL(18,2)	Minimum required balance	NOT NULL
AccountID	INT	Linked account	FK, UNIQUE → Account(AccountID), NOT NULL

Relationer: SavingsAccountDetail → Account (1:1 optional)

3.13 Currency

Purpose: Currency codes and exchange rates.

Column	Datatype	Description	Constraint
CurrencyID	INT	Unique currency ID	PK, Identity(1,1), NOT NULL
Code	CHAR(3)	ISO currency code	NOT NULL, UNIQUE
Name	VARCHAR(100)	Currency name	NOT NULL
ExchangeRate	DECIMAL(18,6)	Rate against base currency	NOT NULL
RateDate	DATE	Date of this rate	NOT NULL

Relationer: Currency → Account (1:N), Currency → Transaction (1:N), Currency → Loan (1:N)

3.14 Card

Purpose: Payment cards linked to accounts.

Column	Datatype	Description	Constraint
CardID	INT	Unique card ID	PK, Identity(1,1), NOT NULL
CardNumber	VARCHAR(19)	PAN	NOT NULL, UNIQUE
CVV	CHAR(4)	Security code	NOT NULL
IssuedDate	DATE	Issuance date	NOT NULL
ExpiryMonth	TINYINT	Expiry month	NOT NULL
ExpiryYear	SMALLINT	Expiry year	NOT NULL
CardTypeID	INT	Card type	FK → CardType(CardTypeID), NOT NULL
AccountID	INT	Linked account	FK → Account(AccountID), NOT NULL

Relationer: Card → Account (M:1), Card → CardType (M:1), Card → CardTransaction (1:N)

3.15 CardType

Purpose: Lookup of card types (Visa, MasterCard...).

Column	Datatype	Description	Constraint
CardTypeID	INT	Unique card type ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Type name	NOT NULL, UNIQUE

Relationer: CardType → Card (1:N)

3.16 CardTransaction

Purpose: Details for card-based transactions (strict 1–0..1 to Transaction).

Column	Datatype	Description	Constraint
CardTransactionID	BIGINT	Unique card transaction ID	PK, Identity(1,1), NOT NULL
TransactionID	BIGINT	Linked general transaction	FK, UNIQUE → Transaction(TransactionID), NOT NULL
CardID	INT	The card used	FK → Card(CardID), NOT NULL
PostedDate	DATETIME	Posting timestamp	NOT NULL
Amount	DECIMAL(18,2)	Transaction amount	NOT NULL
MerchantID	INT	Merchant	FK → Merchant(MerchantID), NOT NULL

Relationer: CardTransaction → Transaction (0..1:1), CardTransaction → Card (M:1), CardTransaction → Merchant (M:1)

3.17 Transaction

Purpose: General ledger of transactions.

Column	Datatype	Description	Constraint
TransactionID	BIGINT	Unique transaction ID	PK, Identity(1,1), NOT NULL
TransactionDate	DATETIME	When transaction occurred	NOT NULL
Amount	DECIMAL(18,2)	Amount	NOT NULL
Description	VARCHAR(255)	Free-text description	NULLABLE
MerchantID	INT	Merchant	FK → Merchant(MerchantID), NULLABLE
PayeeID	INT	External payee	FK → Payee(PayeeID), NULLABLE
AccountID	INT	Related account	FK → Account(AccountID), NOT NULL
TransactionTypeID	INT	Type of transaction	FK → TransactionType(TransactionTypeID), NOT NULL
CurrencyID	INT	Currency	FK → Currency(CurrencyID), NOT NULL

Relationer: Transaction → Account (M:1), Transaction → TransactionType (M:1), Transaction → Currency (M:1), Transaction → Merchant (M:1), Transaction → Payee (M:1), Transaction → CardTransaction (1:0..1)

3.18 TransactionType

Purpose: Lookup table for transaction categories.

Column	Datatype	Description	Constraint
TransactionTypeID	INT	Unique type ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Type name	NOT NULL, UNIQUE

Relationer: TransactionType → Transaction (1:N)

3.19 Merchant

Purpose: Stores merchants where cards/payments occur.

Column	Datatype	Description	Constraint
MerchantID	INT	Unique merchant ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(200)	Merchant name	NOT NULL
MerchantCategoryID	INT	Category of merchant	FK → MerchantCategory(MerchantCategoryID), NOT NULL

Relationer: Merchant → MerchantCategory (M:1), Merchant → Transaction (1:N), Merchant → CardTransaction (1:N)

3.20 MerchantCategory

Purpose: Categories for merchants (e.g. Retail, Services).

Column	Datatype	Description	Constraint
MerchantCategoryID	INT	Unique category ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Category name	NOT NULL, UNIQUE

Relationer: MerchantCategory → Merchant (1:N)

3.21 Loan

Purpose: Tracks loan agreements.

Column	Datatype	Description	Constraint
LoanID	BIGINT	Unique loan ID	PK, Identity(1,1), NOT NULL
PrincipalAmount	DECIMAL(18,2)	Original loan amount	NOT NULL
InterestRate	DECIMAL(5,2)	Annual interest rate (%)	NOT NULL
TermMonths	INT	Duration in months	NOT NULL
StartDate	DATE	Loan start date	NOT NULL
EmployeeID	INT	Loan officer	FK → Employee(EmployeeID), NOT NULL
CurrencyID	INT	Currency	FK → Currency(CurrencyID), NOT NULL
LoanTypeID	INT	Type of loan	FK → LoanType(LoanTypeID), NOT NULL

Relationer: Loan → Employee (M:1), Loan → Currency (M:1), Loan → LoanType (M:1), Loan → CustomerLoan (1:N), Loan → LoanPayment (1:N)

3.22 LoanType

Purpose: Defines types of loans (e.g. Mortgage).

Column	Datatype	Description	Constraint
LoanTypeID	INT	Unique type ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Type name	NOT NULL, UNIQUE

Relationer: LoanType → Loan (1:N)

3.23 CustomerLoan

Purpose: Bridge table for customers and loans (M:N).

Column	Datatype	Description	Constraint
CustomerLoanID	BIGINT	Unique customer-loan record ID	PK, Identity(1,1), NOT NULL
CustomerID	INT	The customer	FK → Customer(CustomerID), NOT NULL
LoanID	BIGINT	The loan	FK → Loan(LoanID), NOT NULL
CustomerLoanRoleID	INT	Role of customer on loan	FK → CustomerLoanRole(CustomerLoanRoleID), NOT NULL

Relationer: CustomerLoan → Customer (M:1), CustomerLoan → Loan (M:1), CustomerLoan → CustomerLoanRole (M:1)

3.24 CustomerLoanRole

Purpose: Defines roles in a customer-loan relationship.

Column	Datatype	Description	Constraint
CustomerLoanRoleID	INT	Unique role ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Role name	NOT NULL, UNIQUE

Relationer: CustomerLoanRole → CustomerLoan (1:N)

3.25 LoanPayment

Purpose: Records payments against loans.

Column	Datatype	Description	Constraint
LoanPaymentID	BIGINT	Unique payment ID	PK, Identity(1,1), NOT NULL
PaymentDate	DATE	Date of payment	NOT NULL
PaymentAmount	DECIMAL(18,2)	Amount paid	NOT NULL
RemainingBalance	DECIMAL(18,2)	Balance left after pay	NOT NULL
LoanID	BIGINT	Loan	FK → Loan(LoanID), NOT NULL

Relationer: LoanPayment → Loan (M:1)

3.26 Employee

Purpose: Bank employee records.

Column	Datatype	Description	Constraint
EmployeeID	INT	Unique employee ID	PK, Identity(1,1), NOT NULL
FirstName	VARCHAR(100)	First name	NOT NULL
LastName	VARCHAR(100)	Last name	NOT NULL
BirthDate	DATE	Date of birth	NOT NULL
Email	VARCHAR(255)	Work email	NOT NULL
PhoneNumber	VARCHAR(50)	Contact number	NULLABLE
HireDate	DATE	Hiring date	NOT NULL
CreatedDate	DATETIME	Record creation timestamp	NOT NULL
GenderID	INT	FK → Gender(GenderID)	NOT NULL
BranchID	INT	FK → Branch(BranchID)	NOT NULL
AddressID	INT	FK → Address(AddressID)	NOT NULL
EmployeeRoleID	INT	FK → EmployeeRole(EmployeeRoleID)	NOT NULL
ManagerID	INT	FK → Employee(EmployeeID) – self-ref	NULLABLE

Relationer: Employee → Address (M:1), Employee → EmployeeRole (M:1), Employee → Employee (0..1:1 self-ref), Employee → Loan (1:N)

3.27 EmployeeRole

Purpose: Lookup for employee roles (Teller, Manager...).

Column	Datatype	Description	Constraint
EmployeeRoleID	INT	Unique role ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(100)	Role name	NOT NULL, UNIQUE

Relationer: EmployeeRole → Employee (1:N)

3.28 Gender

Purpose: Lookup of genders.

Column	Datatype	Description	Constraint
GenderID	INT	Unique gender ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(50)	Gender label	NOT NULL, UNIQUE

Relationer: Gender → Customer (1:N), Gender → Employee (1:N)

3.29 Payee

Purpose: External payees for transfers.

Column	Datatype	Description	Constraint
PayeeID	INT	Unique payee ID	PK, Identity(1,1), NOT NULL
Name	VARCHAR(200)	Payee name	NOT NULL
AccountNumber	VARCHAR(50)	Payee account number	NULLABLE
SWIFTCode	VARCHAR(20)	Payee SWIFT/BIC code	NULLABLE
MerchantID	INT	If payee is also a merchant	FK → Merchant(MerchantID), NULLABLE

Relationer: Payee → Transaction (1:N), Payee → Merchant (0..1:1)

4. Relationships

Below is a consolidated list of every relationship in the schema, with its cardinality and type:

Relationship	Cardinality	Notes
Country → Region	1 → *	One country has many regions.
Region → City	1 → *	One region contains many cities.
City → PostalCode	1 → *	One city may have many postal codes.
PostalCode → Address	1 → *	One postal code can apply to many addresses.
Address → Customer	1 → *	One address can belong to many customers.
Address → Branch	1 → *	One address can host many branches (e.g. sub-offices).
Address → Employee	1 → *	One address can be home to many employees.
Branch → Customer	1 → *	One branch serves many customers.
Branch → Employee	1 → *	One branch employs many employees.
Customer ↔ Account via Disposition	* ↔ *	Many-to-many: each customer may hold multiple accounts, each account may have multiple holders. Implemented as bridge table Disposition with role lookup in DispositionRole.
DispositionRole → Disposition	1 → *	One disposition role can apply to many disposition records.
AccountType → Account	1 → *	One account type is assigned to many accounts.
Currency → Account	1 → *	One currency can be used by many accounts.

Relationship	Cardinality	Notes
Account → SavingsAccountDetail	1 → 0..1	Optional one-to-one: only savings accounts have a detail record.
Account → Card	1 → *	One account can have multiple cards.
Account → Transaction	1 → *	One account may appear on many transactions.
CardType → Card	1 → *	One card type applies to many cards.
Card → CardTransaction	1 → *	One card can have many card-specific transactions.
Transaction → CardTransaction	1 → 0..1	Strict one-to-optional-one: a transaction may have exactly one card-detail record.
Currency → Transaction	1 → *	One currency may be used by many transactions.
TransactionType → Transaction	1 → *	One transaction type applies to many transactions.
MerchantCategory → Merchant	1 → *	One merchant category for many merchants.
Merchant → Transaction	1 → *	One merchant can be the payee in many transactions.
Merchant → CardTransaction	1 → *	One merchant can appear in many card-transactions.
Payee → Transaction	1 → *	One payee can be referenced by many transactions.
Customer ↔ Loan via CustomerLoan	* ↔ *	Many-to-many: each customer may take multiple loans, each loan may have multiple customers. Bridge CustomerLoan with role lookup in CustomerLoanRole.
CustomerLoanRole → CustomerLoan	1 → *	One customer-loan role can apply to many bridge records.
LoanType → Loan	1 → *	One loan type for many loans.

Relationship	Cardinality	Notes
Currency → Loan	1 → *	One currency for many loans.
Employee → Loan	1 → *	One employee (officer) may underwrite many loans.
Loan → LoanPayment	1 → *	One loan may have many payment records.
EmployeeRole → Employee	1 → *	One role type for many employees.
Gender → Customer	1 → *	One gender label for many customers.
Gender → Employee	1 → *	One gender label for many employees.
Employee → Employee (self-reference via ManagerID)	1 → 0..*	One manager can supervise zero or many employees.

5. Normalization & Notes

5.1 Third Normal Form (3NF) Compliance

Table	3NF Justification
Customer	PK = CustomerID; all other attributes (FirstName, LastName, NationalID, BirthDate, etc.) depend solely on CustomerID.
Branch	PK = BranchID; BranchName, CreatedDate, PhoneNumber, SWIFTCode, AddressID are all directly about the branch.
Address	PK = AddressID; Street and HouseNumber depend on AddressID. PostalCodeID is an FK, no transitive dependencies.
PostalCode	PK = PostalCodeID; Code depends on PostalCodeID. CityID is an FK, no non-key attributes depend on CityID besides the direct link.
City	PK = CityID; Name depends on CityID. RegionID is an FK, no derived or multi-valued columns.
Region	PK = RegionID; Name depends on RegionID. CountryID is an FK, no transitive attributes.
Country	PK = CountryID; Name and ISOCode depend on CountryID. ISOCode is uniquely constrained.
Disposition	PK = DispositionID; holds only FKs (CustomerID, AccountID, DispositionRoleID)—no non-key attributes, and all FKs form the relation.
DispositionRole	PK = DispositionRoleID; Name depends solely on that key and is unique.
Account	PK = AccountID; AccountNumber, IBAN, SWIFTCode, Balance, CreatedDate, AccountTypeID, CurrencyID all depend directly on AccountID.
AccountType	PK = AccountTypeID; Name is the only non-key attribute and is unique.

Table	3NF Justification
SavingsAccountDetail	PK = SavingsAccountID; InterestRate and MinimumBalance depend on it. AccountID is FK+UNIQUE—no partial dependencies.
Currency	PK = CurrencyID; Code, Name, ExchangeRate, RateDate all depend solely on CurrencyID.
Card	PK = CardID; CardNumber, CVV, IssuedDate, ExpiryMonth, ExpiryYear, CardTypeID, AccountID all depend directly on CardID.
CardType	PK = CardTypeID; Name depends solely on that key and is unique.
CardTransaction	PK = CardTransactionID; TransactionID is FK+UNIQUE, other attributes (PostedDate, Amount, MerchantID) depend on the PK.
Transaction	PK = TransactionID; all other fields (TransactionDate, Amount, Description, FK links) depend only on TransactionID.
TransactionType	PK = TransactionTypeID; Name depends solely on the key and is unique.
Merchant	PK = MerchantID; Name and MerchantCategoryID depend only on MerchantID.
MerchantCategory	PK = MerchantCategoryID; Name depends solely on the key and is unique.
Loan	PK = LoanID; PrincipalAmount, InterestRate, TermMonths, StartDate, EmployeeID, CurrencyID, LoanTypeID all depend on LoanID.
LoanType	PK = LoanTypeID; Name depends solely on the key and is unique.
CustomerLoan	PK = CustomerLoanID; only FKs (CustomerID, LoanID, CustomerLoanRoleID) – no non-key attributes, so fully normalized.
CustomerLoanRole	PK = CustomerLoanRoleID; Name depends solely on that key and is unique.
LoanPayment	PK = LoanPaymentID; PaymentDate, PaymentAmount, RemainingBalance, LoanID all depend directly on LoanPaymentID.
Employee	PK = EmployeeID; FirstName, LastName, BirthDate, Email,

Table	3NF Justification
	PhoneNumber, HireDate, CreatedDate, AddressID, EmployeeRoleID, ManagerID depend on EmployeeID only.
EmployeeRole	PK = EmployeeRoleID; Name depends solely on the key and is unique.
Gender	PK = GenderID; Name depends solely on the key and is unique.
Payee	PK = PayeeID; Name, AccountNumber, SWIFTCode, MerchantID depend only on PayeeID.

5.2 Many-to-Many Relationships

- Customer ↔ Account via Disposition
 - Bridge table Disposition(CustomerID, AccountID, RoleID) has a composite PK (or its own surrogate PK + unique (CustomerID,AccountID)).
 - No non-key attributes in the bridge, so it's in 3NF.
- Customer ↔ Loan via CustomerLoan
 - Same reasoning as above.

5.3 One-to-One (0..1) Extensions

- Account → SavingsAccountDetail
 - SavingsAccountDetail.AccountID is both FK and UNIQUE. No partial dependencies.
- Transaction → CardTransaction
 - CardTransaction.TransactionID is FK+UNIQUE. Specialized fields (CVV, PostedDate) live here, keeping Transaction lean.

5.4 Additional Notes

- **Indexing:** All FKs should be indexed for performance (recommend creating non-clustered indexes).
 - **Naming Conventions:** English singular table names, PascalCase columns.
 - **Constraints:** All business-critical columns (e.g. NationalID, AccountNumber) are marked UNIQUE.
 - **Future Enhancements:** Consider history/audit tables for compliance, value-history for exchange rates, etc.
-